

Django의 tests.py란?

- Django 프로젝트에서 자동화된 테스트 코드를 작성하는 파일입니다.

왜 필요할까?

- 사람이 직접 버튼을 누르거나 API를 호출해서 확인하는 대신, 코드로 기능이 잘 동작하는지 검사할 수 있음 → 버그 예방, 유지보수 편리

특징

- Python의 unittest 기반
 - Django의 테스트는 기본적으로 파이썬 내장 라이브러리인 `unittest` 를 확장해서 사용
- 테스트 클래스와 메소드 작성
 - `TestCase` 클래스를 상속받아 작성
 - 함수 이름을 `test_` 로 시작하면 테스트로 인식
- 테스트 실행 명령어

```
python manage.py test
```

프로젝트 생성

단계1: 모듈 추가 설치

```
pip install django  
pip install djangorestframework  
pip install django-cors-headers  
pip install PyJWT
```

단계2: 프로젝트 & 앱 생성

```
django-admin startproject config .  
python manage.py startapp api
```

단계3: settings.py

- 앱 추가

```
INSTALLED_APPS = [  
    ...,  
    "rest_framework",  
    "corsheaders",  
    "api"  
]
```

- CORS 크로스 도메인 설정

```
MIDDLEWARE = [  
    "corsheaders.middleware.CorsMiddleware", # 맨위에 추가!!  
    ...  
]  
  
CORS_ORIGIN_ALLOW_ALL = True  
CORS_ALLOW_CREDENTIALS = True
```

- Django REST Framework 설정

```
REST_FRAMEWORK = {  
    'EXCEPTION_HANDLER': 'api.exceptions.status_code_handler'  
}
```

단계4: serializers

- 생성: api.serializers.py

```
from rest_framework import serializers
from django.contrib.auth.models import User

class UserSerializer(serializers.ModelSerializer):
    class Meta:
        model = User
        fields = ['id', 'username', 'email', 'password']
        ...
```

단계5: exceptions 처리

- 생성: api.exceptions.py

```
from rest_framework.views import exception_handler

def status_code_handler(exc, context):
    response = exception_handler(exc, context)

    if response is not None:
        response.status_code = 777
    ...
```

단계6: Json Web Token

- 생성: api.authentication.py

```
import jwt
import datetime
import enum

from rest_framework.exceptions import AuthenticationFailed

class JWT_KEY(enum.Enum):
    RANDOM_OF_ACCESS_KEY = (enum.auto(), 'access_secret', datetime.timedelta(seconds=120), 'HS256', '랜덤한 조합의 키')
    RANDOM_OF_REFRESH_KEY = (enum.auto(), 'refresh_secret', datetime.timedelta(days=2), 'HS256', '랜덤한 조합의 키')
    ...
```

단계7: api.views.py

```
from rest_framework.authentication import get_authorization_header
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework.exceptions import AuthenticationFailed , APIException

from django.contrib.auth.models import User

from .serializers import UserSerializer
from .authentication import create_access_token, create_refresh_token, decode_access_token, decode_refresh_token

# Create your views here.
class Register(APIView):
    def post(self, request):
        serializer = UserSerializer(data=request.data)
        ...
```

단계8: api.urls.py

```
from django.urls import path
from .views import Register, LoginView, HelloWorldView, RefreshView, Logoutview

urlpatterns = [
    path('register/', Register.as_view(), name='register'),
    path('login/', LoginView.as_view(), name='login'),
    path('hello-world/', HelloWorldView.as_view(), name='hello-world'),
    path('refresh/', RefreshView.as_view(), name='refresh'),
    ...
]
```

단계9: config.urls.py

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path("admin/", admin.site.urls),
    path("api/", include("api.urls"))
]
```

실행

단계1: 마이그레이션(migration)

```
$ (.venv) python manage.py makemigrations
```

단계2: Database에 마이그레이션 정보 반영

```
$ (.venv) python manage.py migrate
```

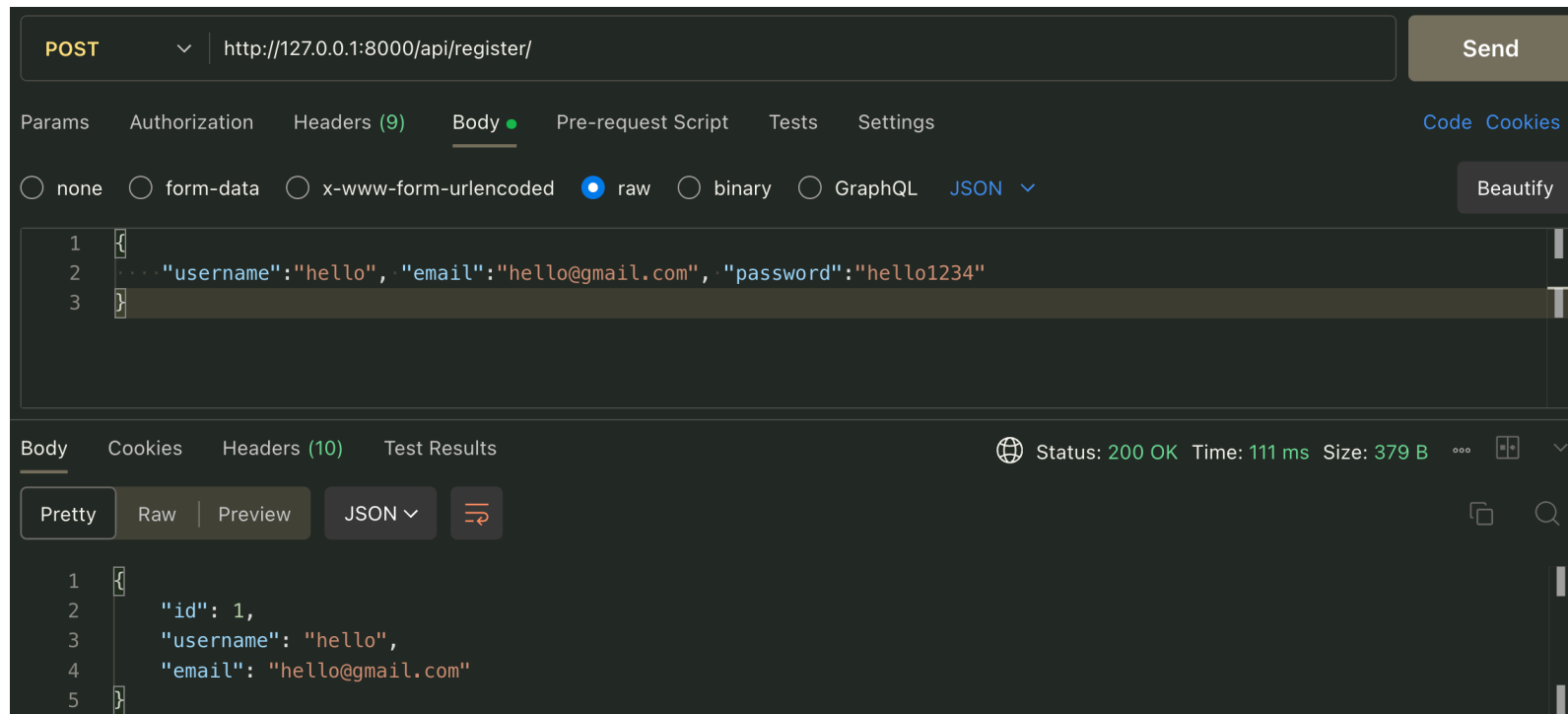
단계3: server 실행 및 관리자 페이지 접속

```
$ (.venv) python manage.py runserver
```

단계4: register

- <http://127.0.0.1:8000/api/register/>

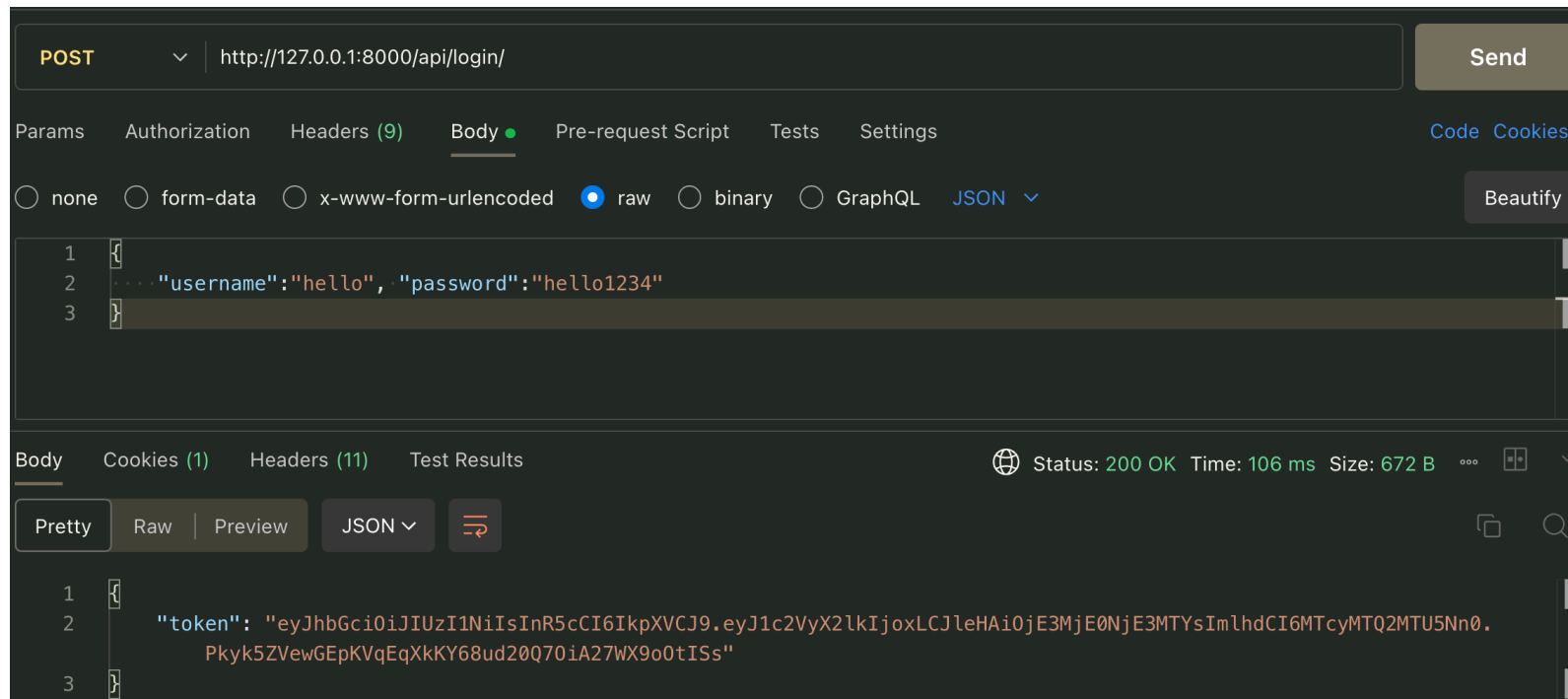
```
{  
  "username": "hello", "email": "hello@gmail.com", "password": "hello1234"  
}
```



단계5: login

- <http://127.0.0.1:8000/api/login/>

```
{  
  "username": "hello", "password": "hello1234"  
}
```



test

단계1: api.tests.py

- 테스트 함수들이 실행하기 전에 호출되는 함수
- 테스트하기전에 필요한 변수들을 미리 생성

```
def setUp(self) -> None:
    self.user = User.objects.create_user("tester", "test@email.com", "tester")
    self.client = APIClient()
    self.login_url = reverse('login')
    ...
```

- login test

```
def test_login_return_jwt(self):  
    """  
    The login view return an access token and a refresh token  
    """  
    credentials = {  
        "username": "tester",  
        "password": "tester"  
    }  
  
    response = self.client.post(self.login_url, credentials)  
    ...
```

- bad login test

```
def test_exceptions_with_bad_login(self):  
    """  
    The login view doesn't return tokens if we use bad credentials  
    """  
    credentials = {  
        "username": "bad tester",  
        "password": "bad tester"  
    }  
    ...
```

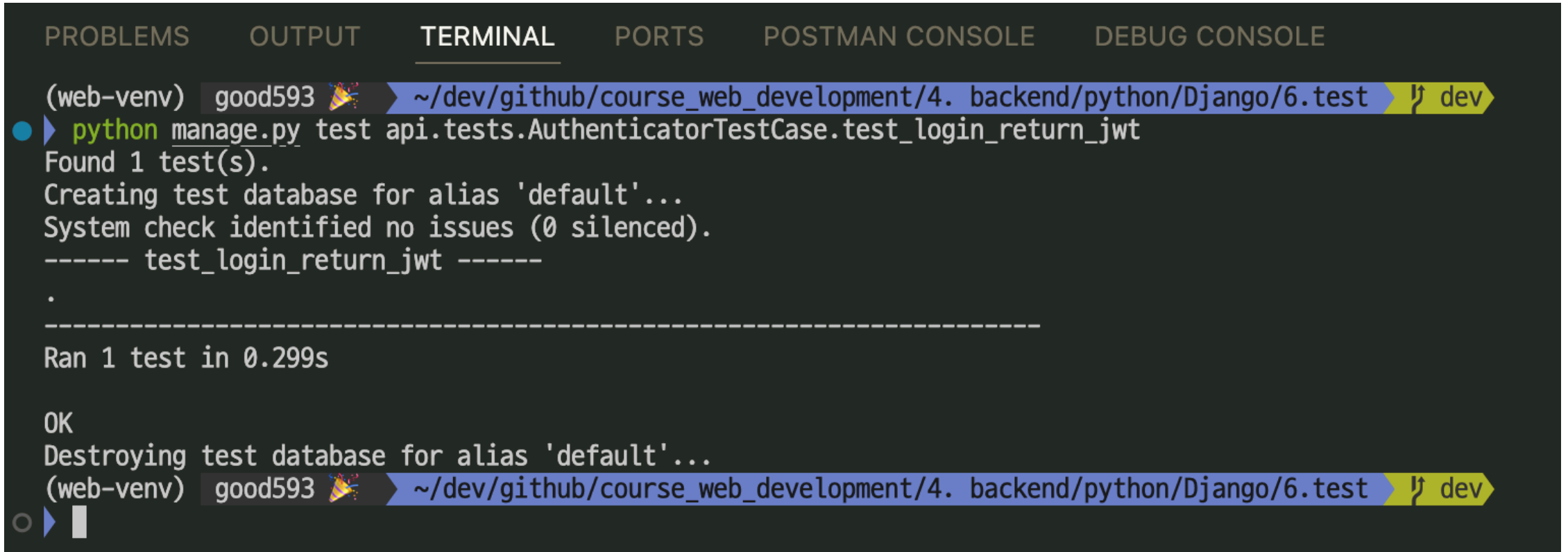
- jwt test

```
def test_with_jwt_return(self):  
  
    self.client.credentials(HTTP_AUTHORIZATION='Bearer '+self.access_token)  
    response = self.client.get(self.hello_world_url)  
    print("----- test_with_jwt_return -----")  
    print(response.status_code)  
    print(response.json())  
    ...
```

단계2: run tests.py

- test_login_return_jwt 실행

```
python manage.py test api.tests.AuthenticatorTestCase.test_login_return_jwt
```



The screenshot shows a VS Code terminal window with the 'TERMINAL' tab selected. The terminal displays the output of running the command `python manage.py test api.tests.AuthenticatorTestCase.test_login_return_jwt` in a virtual environment named 'web-venv'. The output shows that one test was found and passed successfully. The terminal also shows the creation and destruction of a test database for the alias 'default'.

```
PROBLEMS OUTPUT TERMINAL PORTS POSTMAN CONSOLE DEBUG CONSOLE
(web-venv) good593 ▶ ~/dev/github/course_web_development/4. backend/python/Django/6.test ▶ dev
▶ python manage.py test api.tests.AuthenticatorTestCase.test_login_return_jwt
Found 1 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
----- test_login_return_jwt -----
.
-----
Ran 1 test in 0.299s

OK
Destroying test database for alias 'default'...
(web-venv) good593 ▶ ~/dev/github/course_web_development/4. backend/python/Django/6.test ▶ dev
▶
```

- 전체 테스트 함수 실행

```
python manage.py test api.tests.AuthenticatorTestCase
```

```
(web-venv) good593 ▶ ~/dev/github/course_web_development/4. backend/python/Django/6.test ▶ dev
▶ python manage.py test api.tests.AuthenticatorTestCase
Found 3 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
----- test_exceptions_with_bad_login -----
777
{'detail': 'User not found'}
.----- test_login_return_jwt -----
.----- test_with_jwt_return -----
200
{'message': 'Hello World'}
.
-----
Ran 3 tests in 0.709s

OK
Destroying test database for alias 'default'...
(web-venv) good593 ▶ ~/dev/github/course_web_development/4. backend/python/Django/6.test ▶ dev
▶
```

Swagger

- API를 문서화하고 테스트할 수 있게 해주는 도구/프레임워크예요.
- REST API 서버를 만들었을 때, 어떤 주소로 어떤 요청을 보내야 하고, 어떤 응답이 오는지 쉽게 확인할 수 있습니다.

주요 기능

- 자동 문서화
 - API의 요청/응답 형식, 파라미터, 예시 등을 보기 좋게 웹 UI로 보여줌
- 실시간 테스트
 - 브라우저에서 바로 API 요청을 실행해볼 수 있음
 - 예: `POST /login` 버튼 눌러서 JSON 응답 확인
- 팀 협업에 유용
 - 프론트엔드/모바일 개발자들이 API 문서를 보면서 바로 개발 가능
 - 백엔드와의 커뮤니케이션 비용을 크게 줄여줌

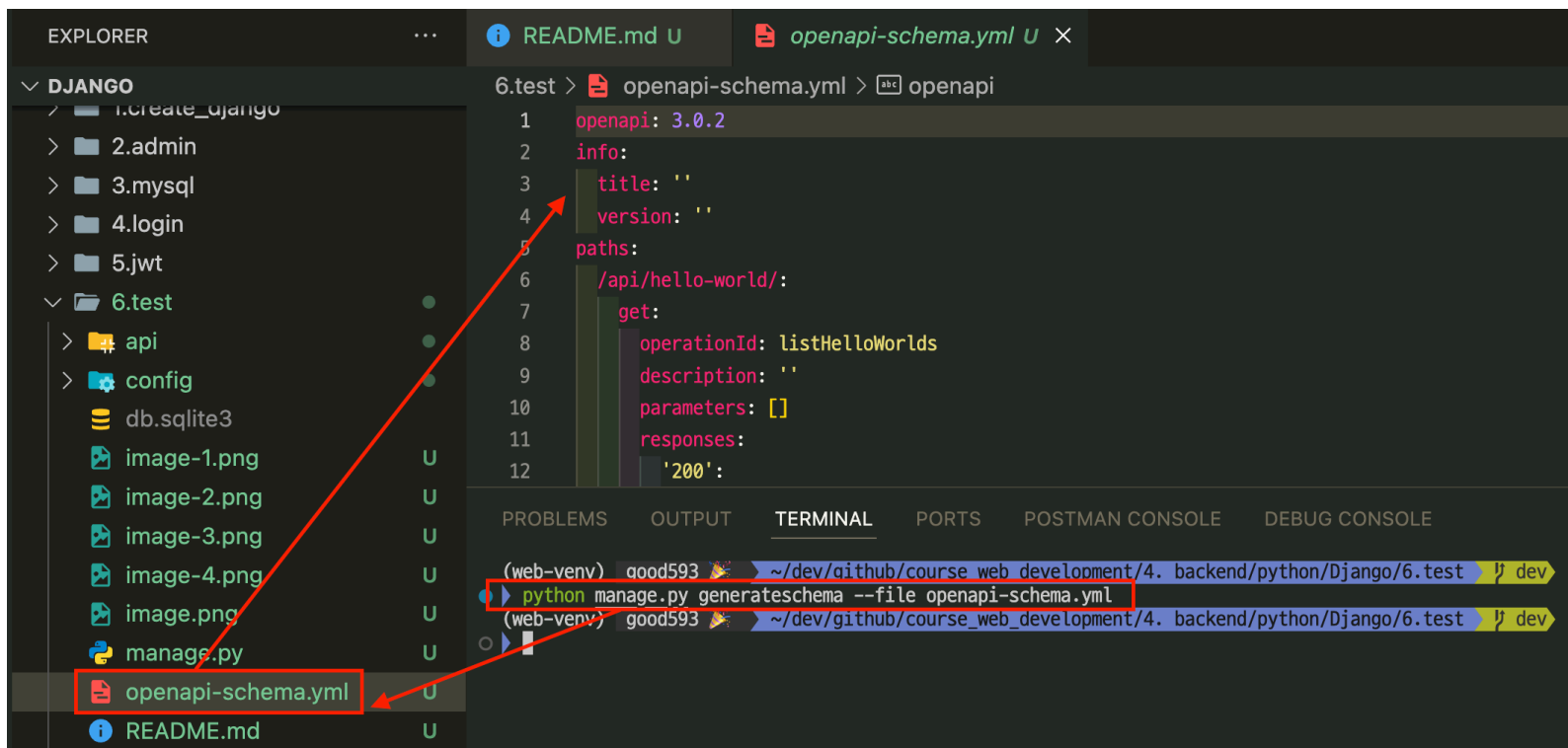
단계1: install

```
pip install pyyaml uritemplate
```

```
(web-venv) good593 ➤ ~/dev/github/course_web_development/4. backend/python/Django/6.test ➤ ↵ dev
● ➤ pip install pyyaml uritemplate
Requirement already satisfied: pyyaml in /Users/gyoungwon-cho/.pyenv/versions/web-venv/lib/python3.11/site-packages (6.0)
Requirement already satisfied: uritemplate in /Users/gyoungwon-cho/.pyenv/versions/web-venv/lib/python3.11/site-packages (4.1.1)
(web-venv) good593 ➤ ~/dev/github/course_web_development/4. backend/python/Django/6.test ➤ ↵ dev
○ ➤ █
```

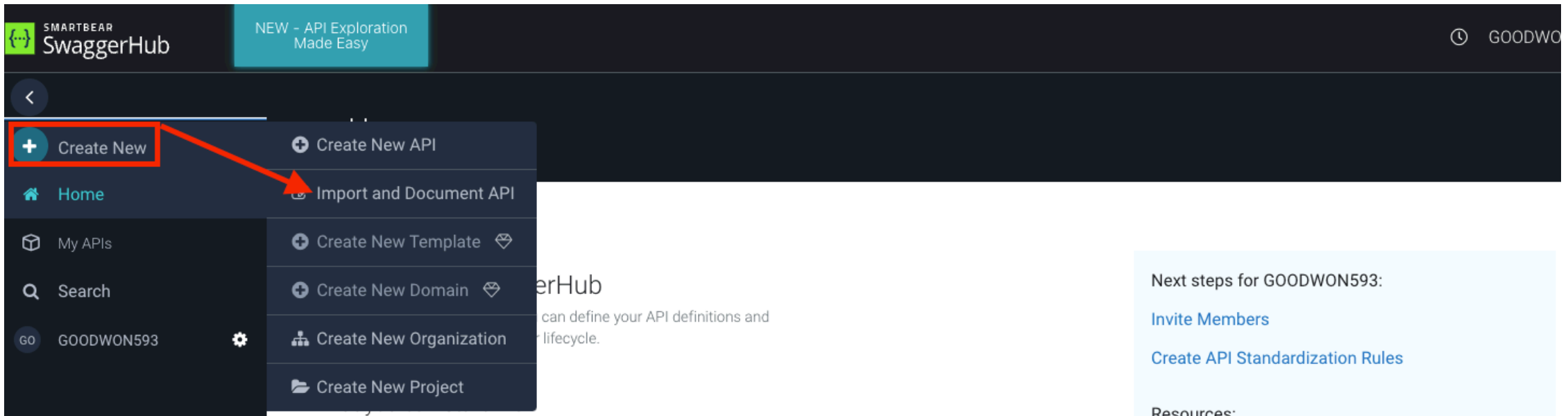
단계2: openapi-schema.yml 생성

```
python manage.py generateschema --file openapi-schema.yml
```



단계3: SwaggerHub 적용

- 로그인 후 사용



- upload openapi-schema.yml

your API defin

Import API

Enter the path or URL to your API JSON or YAML spec, or browse to select a local file. Import to your personal account or change the Owner to any of your organizations. [Learn more](#)

Path or URL

openapi-schema.yml

X

Owner

GOODWON593

▼

Project

--- None ---

▼

Visibility

Public

▼

i

1/3 public APIs used. [Upgrade](#)

Cancel

Upload File

Rules

on

PI (Swagger)

Documentation

Testing

- API Exploration

- import api

API definition

Import API

Update the name and version below, or keep the values from the spec. [Learn more](#)

Name

django-test

Version

1.0.0

✓ Success. The resource is valid YAML

Cancel

Import Definition

Rules

on

API (Swagger)

Documentation

Testing

- API Exploration

Info

Tags

Servers

Search

api ▾

- GET /api/hello-world/
- POST /api/register/
- POST /api/login/
- POST /api/refresh/
- POST /api/logout/

Schemas

Aa ⚙️

SAVE ▾

PACTFLOW

Sync

```
1 openapi: 3.0.2
2 info:
3   title: ''
4   version: 1.0.0
5 paths:
6   /api/hello-world/:
7     get:
8       operationId: listHelloWorlds
9       description: ''
10      parameters: []
11      responses:
12        '200':
13          content:
14            application/json:
15              schema:
16                type: array
17                items: {}
18              description: ''
19      tags:
20        - api
21   /api/register/:
22     post:
23       operationId: createRegister
24       description: ''
25       parameters: []
26       requestBody:
27         content:
28           application/json:
29             schema: {}
30           application/x-www-form-urlencoded:
31             schema: {}
32           multipart/form-data:
33             schema: {}
```

1.0.0 OAS 3.0

Note: "Try it out" is disabled because no servers are specified in the "servers" array. Please see: [info on OAS3 servers](#)

api

GET /api/hello-world/ 📄 ⬅️ 🔗 ▾

POST /api/register/ 📄 ⬅️ 🔗 ▾

POST /api/login/ 📄 ⬅️ 🔗 ▾

POST /api/refresh/ 📄 ⬅️ 🔗 ▾

POST /api/logout/ 📄 ⬅️ 🔗 ▾

SSL certificates are validated! Do not validate 🔒